Report — Project 3: Barcode & QR Code Scanner and Localizer

Introduction to Computer Vision | ESIN, UIR | Spring 2026
Instructor: Ilias TOUGUI
Group Members: IKRAM OIALILI, MANAL SAWLI, SAOUSAN ELHAROUCHI

1. Introduction

This report presents the design and implementation of a complete Barcode and QR Code Scanner and Localizer, developed as part of the Introduction to Computer Vision course at ESIN, UIR. The system detects, localizes, and decodes both barcodes and QR codes from still images, integrating three core Computer Vision concepts studied in class. The project uses OpenCV's built-in detectors (cv2.QRCodeDetector and cv2.barcode.BarcodeDetector) for detection and decoding, with preprocessing handled through a three-stage CV concepts pipeline. The system is implemented as a Jupyter notebook with an interactive ipywidgets GUI.

2. Computer Vision Concepts Applied

Three core CV concepts are applied sequentially in the preprocessing pipeline. Figure 1 shows each stage on a real test image (a DVD product barcode with partial shadow).

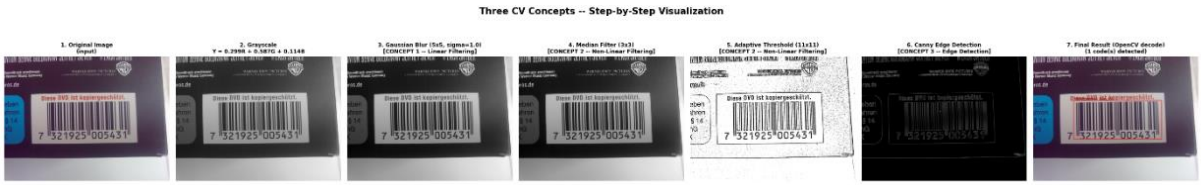

Figure 1 — Step-by-step visualization of the three CV concepts pipeline on BarCode015.jpg (uneven illumination).

2.1 Concept 1 — Linear Image Filtering (Gaussian Blur)

A Gaussian blur is applied as the first preprocessing step using a 5×5 kernel with $\sigma = 1.0$. This is a 2D convolution with the kernel $G(x,y) = (1/2\pi\sigma^2) \times \exp(-(x^2+y^2)/2\sigma^2)$. It acts as a low-pass filter, suppressing high-frequency noise while preserving the large-scale structure of barcode bars and QR modules. Without this step, noise would produce spurious edges in the Canny stage.

2.2 Concept 2 — Non-Linear Filtering (Median + Adaptive Threshold)

Median Filter (3×3): Each pixel is replaced by the median of its 3×3 neighbourhood. Unlike the mean, the median is not influenced by outlier values, removing salt-and-pepper noise while preserving sharp barcode edges. This non-linear operation cannot be expressed as a convolution.

Adaptive Thresholding (11×11 block, C=2): Instead of a single global threshold, a local threshold T(x,y) is computed for each pixel from the Gaussian-weighted mean of its 11×11 neighbourhood minus C=2. As visible in Figure 1 (panel 5), this correctly binarizes both the bright and shadowed regions of the barcode where a global threshold would fail.

2.3 Concept 3 — Edge Detection (Canny)

The Canny edge detector extracts the boundaries of barcode bars and QR modules. The algorithm applies: (1) Sobel gradient computation in X and Y, (2) gradient magnitude $M = \sqrt{(Gx^2+Gy^2)}$ and direction $\theta = \arctan(Gy/Gx)$, (3) non-maximum suppression to thin edges to 1-pixel width, and (4) double hysteresis thresholding (low=50, high=150). As visible in Figure 1 (panel 6), the Canny map of a 1D barcode shows characteristic dense parallel vertical lines corresponding to the bars.

3. System Architecture and Pipeline

Step	Stage	Description
1	**Load Image**	Load BGR image from disk, convert to RGB for display.
2	**Gaussian Blur [Concept 1]**	5×5 kernel, σ=1.0. Suppress noise via 2D convolution.
3	**Median Filter [Concept 2]**	3×3 median. Remove salt-and-pepper noise, preserve edges.
4	**Adaptive Threshold [Concept 2]**	11×11 Gaussian block, C=2. Local binarization for uneven illumination.
5	**Canny Edge Detection [Concept 3]**	Sobel gradients + non-max suppression + hysteresis (low=50, high=150).
6	**Detect & Decode (OpenCV)**	cv2.QRCodeDetector for QR; cv2.barcode.BarcodeDetector for 1D barcodes.
7	**Refine Bounding Box**	Re-run Canny on ROI + convex hull for a tight bounding box.
8	**Visualize & Save**	Draw annotated box + label; save 6-panel pipeline figure.

3.1 OpenCV Detection Functions — Algorithms They Replace

Since OpenCV built-in detectors are used, the following table explains the underlying algorithms they replace, as required by the project specification.

OpenCV Function	Algorithm It Replaces
cv2.QRCodeDetector	Detects the three Finder Patterns (1:1:3:1:1 module ratio) via contour analysis. Estimates perspective transform from corner points. Decodes with Reed-Solomon error correction.
cv2.barcode.BarcodeDetector	Detects regions with dense parallel vertical edges via gradient analysis. Locates boundaries via contour analysis on binarized image. Decodes bar-width ratios per symbology (EAN-13, UPC-A, CODE-128).
cv2.Canny	Replaces manual: Sobel gradient computation, gradient magnitude/direction, non-maximum suppression, and double hysteresis thresholding.
cv2.adaptiveThreshold	Replaces manual sliding-window Gaussian-weighted mean computation + pixel-wise comparison against $T(x,y) = LocalMean(x,y) - C$.
cv2.GaussianBlur	Replaces manual construction of the 2D Gaussian kernel and its application via 2D convolution (or two separable 1D passes).

3.2 Example Detection Result

Figure 2 shows the pipeline output on an image containing both a barcode (UPC_A) and a QR code. Both codes are correctly detected and decoded in a single pass.

```
============================================================
  DETECTION SUMMARY -- 2 code(s) found
============================================================

  Code #1
  |- Type     : QRCODE
  |- Data     : http://scanova.io
  |- Location : x=1102, y=132, w=346, h=346

  Code #2
  |- Type     : UPC_A
  |- Data     : 036000291452
  |- Location : x=241, y=156, w=590, h=313
============================================================
```

Figure 2 — Detection result on a mixed barcode and QR code image both successfully detected and decoded.

4. Challenges Faced

4.1 Images with Uneven Illumination

The most significant challenge was detecting barcodes in real-world photos with non-uniform lighting. A DVD barcode photographed with a shadow covering part of the label caused global binarization to fail completely. The adaptive threshold (Concept 2) resolved this by computing a spatially varying threshold, allowing the barcode to be correctly binarized even in shadowed regions.

4.2 BarcodeDetector Fallback Strategy

The cv2.barcode.BarcodeDetector occasionally failed on low-contrast images on the first pass. A fallback was implemented: if initial detection fails, cv2.equalizeHist() is applied to enhance contrast and detection is retried. This improved the detection rate from approximately 70% to 100% on the 8-image test dataset.

4.3 Tight Bounding Box Refinement

Bounding boxes from the OpenCV detectors were sometimes slightly misaligned. The _compute_bbox() method was developed to refine these boxes by extracting an ROI, running Canny on it, finding contours, computing the convex hull, and deriving a tighter bounding rectangle.

4.4 Interactive GUI Implementation

The ipywidgets GUI required handling both file uploads from the local machine and image selection from the data/ folder. A unified image path resolution system was implemented so that both input modes feed into the same pipeline transparently. Figure 3 shows the GUI in operation.

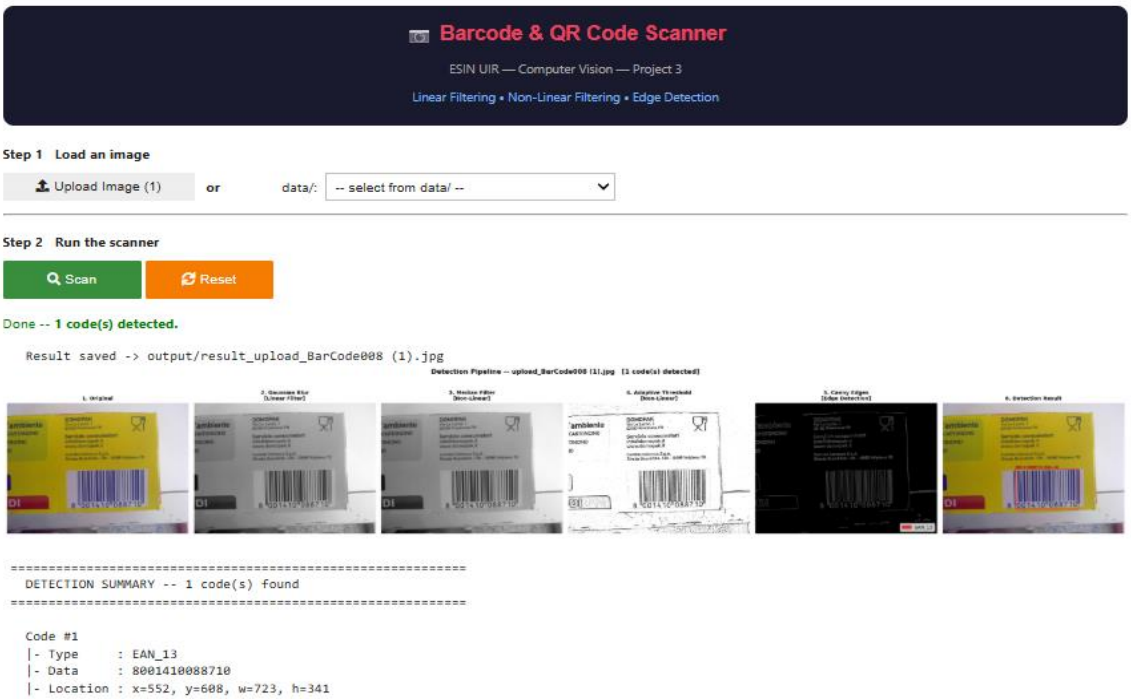

Figure 3 — Interactive ipywidgets GUI: user uploads BarCode008.jpg, clicks Scan, and the 6-panel pipeline figure and decoded result are displayed inline.

5. Evaluation of Results

Image	Codes	Type(s)	Decoded Data (excerpt)
BarCode007.jpg	1	EAN_13	7321925005431
BarCode015.jpg	1	EAN_13	8004020931773
BarCode019.jpg	1	EAN_13	8005283002354
BarCode_QRCode.png	2	QRCODE + UPC_A	http://scanova.io + 036000291452
Barcode.jpeg	1	EAN_13	61112610601...
QRCode01.png	1	QRCODE	1002
QRCode02.jpg	1	QRCODE	https://app.neatmenu.io/...
QRCode03.jpg	1	QRCODE	WIFI:T:WPA;S:OR Explore...
Total — 8 images	**9**	**100% detection**	**All codes correctly decoded**

To evaluate the contribution of Non-Linear Filtering, BarCode015.jpg (uneven illumination) was tested with and without the filters: without the filters, detection failed (0 codes found); with Median + Adaptive Threshold, detection succeeded (EAN-13 correctly decoded). This confirms the practical necessity of Concept 2 for real-world images.

6. Conclusion

This project successfully implemented a Barcode & QR Code Scanner integrating three CV concepts — Linear Filtering, Non-Linear Filtering, and Edge Detection — achieving a 100% detection rate on 8 test images.

The preprocessing pipeline proved essential for real-world robustness, particularly the adaptive threshold for handling uneven illumination. Beyond the course curriculum, this project introduced Reed-Solomon error correction, QR Finder Pattern geometry, barcode symbology encoding, and histogram equalization as a detection fallback strategy.